PROFESSIONAL DEVELOPMENT GUIDE

`title`

7/22/2026

Copyright © 2026 Steve Snively

Contents

CHAPTER 1

The Modern .NET Landscape

Chapter Purpose

This chapter rebuilds the map.

If your working model of .NET was formed around Visual Studio 2019, .NET Framework or early modern .NET, Windows Server, IIS, and SQL Server, the core skills still matter. C# still matters. HTTP still matters. SQL still matters. Production discipline still matters.

What changed is the shape of the system around those skills.

Modern .NET development is no longer centered on one Windows server running one IIS-hosted application connected to one SQL Server. That model still exists, and many businesses still run it successfully, but it is no longer the default mental model for new professional .NET systems. Today, the application is often one part of a larger platform made of source control, automated validation, containers, cloud services, managed identity, deployment pipelines, observability, and sometimes AI services.

The purpose of this chapter is not to teach Docker, Kubernetes, CI/CD, cloud hosting, or AI in detail. Those subjects come later. The purpose is to show where they live on the map, why they became important, and how they relate to the enterprise .NET knowledge you already have.

By the end of this chapter, you should be able to explain how modern .NET systems fit together at a high level and why the ecosystem changed after the Visual Studio 2019 and .NET 5 era.

Where This Fits

This is the first chapter because every later chapter depends on the same orientation: modern .NET is not just a runtime upgrade. It is a different way of thinking about the application lifecycle.

A traditional enterprise .NET system might have looked like this:

```
Developer workstation
        |
        v
Visual Studio build
        |
        v
Manual or scripted deployment
        |
        v
Windows Server + IIS
        |
        v
SQL Server
```

A modern .NET system often looks more like this:

```
Developer workstation or cloud dev environment
        |
        v
```

```
Git repository and pull request
        |
        v
Automated build and test pipeline
        |
        v
Container image or deployable package
        |
        v
Cloud platform, container host, or orchestrator
        |
        v
Managed databases, caches, queues, identity, and AI services
        |
        v
Logs, metrics, traces, alerts, and deployment feedback
```

The application code is still central, but it is no longer the whole story. Modern professional development treats the path from source code to production as part of the system.

This book follows that path. First, it rebuilds the mental model. Then it builds applications. Then it introduces Linux, containers, deployment, cloud platforms, production operations, AI, architecture, and interview preparation.

Connection to the Reader's Existing Model

You already know the old landmarks.

You know that an IIS application pool isolates a web application process. You know that configuration files can change behavior without recompiling. You know that SQL Server is both a database engine and an operational responsibility. You know that Windows services run background work. You know that a load-balanced pair of Windows servers introduces session, deployment, and diagnostic concerns that a single server does not.

Modern .NET keeps many of those ideas but changes where the boundaries are.

An ASP.NET Core process still receives HTTP requests and produces responses, but IIS may be only one possible front end, or not involved at all. A container can package the application and its runtime dependencies in a way that feels a little like a disciplined deployment folder, but it also behaves like a standardized process boundary. A cloud platform can replace some server administration work with managed services, but it introduces new ownership questions around identity, cost, networking, and reliability.

The analogy to traditional Windows hosting is useful because the same concerns remain:

- Where does the process run?
- How is it configured?
- How does it connect to the database?
- How is it deployed?
- How do you know whether it is healthy?
- What happens when it fails?

The analogy breaks down because modern systems distribute those concerns across more tools. Instead of one administrator configuring one server, a team may define build rules in source control, publish a container image, provision infrastructure with code, deploy to a managed platform, and monitor behavior through telemetry.

That sounds like more moving parts because it is. The tradeoff is that those parts make repeatability, scale, automation, and team collaboration easier when the system grows.

Layer 1 — Conceptual Model

The modern .NET landscape is the collection of runtimes, tools, platforms, and operating practices used to build and run .NET applications today.
At the center is .NET itself: the runtime, SDK, base class libraries, languages, ASP.NET Core, and related frameworks. Around it is the professional delivery system: source control, automated tests, containerization, cloud hosting, configuration, secrets, observability, security, and operations.
The important conceptual shift is this:

```
Old center of gravity: the server
New center of gravity: the delivery platform
```

In the older model, the server was often the durable center of the application. You installed software on it, configured IIS, patched the operating system, managed local folders, copied releases into place, and inspected logs on the machine.
In the modern model, the server may be temporary, abstract, replicated, or hidden behind a managed service. The durable center becomes the combination of source code, build pipeline, deployment definition, container image, configuration system, managed data services, and telemetry.
This does not mean servers disappeared. It means developers are expected to understand platforms that create, replace, scale, observe, and secure runtime environments.
Modern .NET exists to let .NET applications participate naturally in that world:

- cross-platform runtime support instead of Windows-only assumptions;
- command-line and SDK-based workflows instead of IDE-only workflows;
- ASP.NET Core hosting that can run behind IIS, Nginx, cloud load balancers, or container platforms;
- container images for repeatable deployment;
- cloud-native configuration, logging, health checks, and dependency injection;
- libraries and abstractions that integrate with modern identity, telemetry, messaging, and AI services.

It does not solve every problem by itself. Modern .NET does not automatically make an application scalable, secure, observable, or maintainable. It gives you the building blocks and platform alignment to do those things deliberately.

Layer 2 — System Relationships

A modern .NET application participates in a larger system. Understanding the relationships is more important at first than memorizing product names.
The source repository is the system of record for code and increasingly for configuration templates, build definitions, deployment manifests, tests, and architecture notes. In older environments, some of that knowledge lived in server settings or deployment runbooks. In modern environments, teams try to move as much of it as practical into versioned artifacts.
The build system converts source code into something runnable. That might be a published folder, a NuGet package, a container image, or a deployable artifact. The build is not just compilation. It usually includes restore, static checks, tests, packaging, and sometimes vulnerability checks.
The runtime environment is where the application process executes. It might be a developer workstation, a Windows service, IIS, a Linux VM, a container host, a platform-as-a-service environment, or Kubernetes. ASP.NET Core is flexible enough to run in several of these environments, but each environment changes the operational responsibilities.
The application dependencies provide the services the code needs to do useful work: SQL Server, caches, queues, file storage, identity providers, external APIs, observability systems, and AI models. In modern systems, more of these dependencies are managed services rather than software installed on the same server as the application.

The deployment system moves validated changes into an environment. A manual copy to an IIS folder is a deployment system, just a fragile one. A modern pipeline makes the steps explicit, repeatable, reviewable, and observable.
The feedback system tells the team what happened after deployment. Logs, metrics, traces, health checks, alerts, dashboards, and error reports are not extras. They are the way a team sees a system that may be spread across multiple processes, services, regions, and managed platforms.
The main failure boundary also changed. In a simple IIS application, the failure boundary was often the web server, application pool, database, or network. Those still matter. Modern systems add more boundaries: container startup, image registry access, cloud identity, service discovery, queue backlogs, rate limits, region outages, deployment pipeline failures, and telemetry gaps.
The professional .NET developer does not need to be the expert owner of every boundary, but they do need to recognize them.

Layer 3 — Core Mechanics

This chapter keeps mechanics small. Later chapters go deep.
The first mechanic is the modern .NET release model. Microsoft now ships major .NET releases annually, and releases alternate between Long Term Support (LTS) and Standard Term Support (STS). As of July 22, 2026, Microsoft lists .NET 10 as the current LTS release, with .NET 9 as an STS release and .NET 8 as an LTS release still in support. This matters because choosing a target framework is also choosing a support and upgrade rhythm.
The second mechanic is side-by-side installation. Modern .NET versions can be installed alongside one another. That is different from the old feeling of a machine-wide .NET Framework version being part of the Windows server's personality. A developer machine, build agent, or server may have multiple SDKs or runtimes installed.
The third mechanic is SDK-centered development. The .NET SDK supplies the command-line tools used to create, build, test, publish, and package projects. Visual Studio remains important, especially for many Windows-based enterprise teams, but the SDK makes the workflow portable across local machines, CI agents, containers, and cloud build environments.
The fourth mechanic is ASP.NET Core's hosting flexibility. An ASP.NET Core application is a process that can run behind different front ends. IIS can still be part of the story, but it is no longer the only natural host. The same application model can run on Windows, Linux, in a container, or on a managed cloud platform.
The fifth mechanic is containerization. A container image packages the application with the runtime environment expected by the application. Microsoft publishes official .NET container images for different scenarios, including SDK images for building and ASP.NET/runtime images for running applications.
The sixth mechanic is platform integration. Modern .NET applications commonly use dependency injection, configuration providers, structured logging, health checks, and telemetry hooks. These are not decorative framework features. They exist because modern applications must be configured, deployed, monitored, and changed repeatedly across environments.
The seventh mechanic is AI integration. AI is now part of the modern .NET landscape, but in this book it is not magic dust sprinkled over every chapter. For now, treat AI as two related developments: AI-assisted development, where tools help developers write and reason about code, and AI-enabled applications, where .NET systems call models, embedding services, or agent frameworks as part of product behavior.

Layer 4 — Developer Workflow

A typical modern .NET workflow is still recognizable.
You create a project. You write C#. You run it locally. You debug it. You test it. You check it into source control. You deploy it. You fix bugs.
The difference is that the workflow is less centered on one workstation and one server.

At a high level, the workflow looks like this:

```
Create or clone repository
        |
        v
Restore dependencies
        |
        v
Run and debug locally
        |
        v
Add tests and commit changes
        |
        v
Open pull request
        |
        v
Automated build and test checks
        |
        v
Package or publish
        |
        v
Deploy to an environment
        |
        v
Observe behavior and iterate
```

In Visual Studio 2019-era development, it was common for the IDE to hide many of these steps. Modern teams still use IDEs, but they also expect the same steps to run outside the IDE. That is why command-line workflows, project files, source-controlled pipeline definitions, and repeatable local setup have become more important.
For this first chapter, the useful commands are discovery commands rather than implementation commands:

```
dotnet --info
dotnet --list-sdks
dotnet --list-runtimes
```

On Windows PowerShell, the same commands apply:

```
dotnet --info
dotnet --list-sdks
dotnet --list-runtimes
```

These commands answer basic orientation questions: which SDKs are installed, which runtimes are available, and what environment the .NET CLI sees.
The details of creating projects, building APIs, testing, containerizing, and deploying come later. For now, the important workflow shift is that modern .NET development is designed to be repeatable across local development, automated builds, and production environments.

Layer 5 — Production Usage

Production is where the modern landscape becomes real.
In a traditional IIS deployment, production concerns were often concentrated on the server:

- IIS configuration;

- application pool identity;
- installed .NET runtime;
- web.config settings;
- Windows Event Log;
- file permissions;
- SQL Server connection strings;
- load balancer behavior.

Modern .NET production systems still have equivalent concerns, but they are distributed across the application, platform, and delivery process.

Configuration is usually environment-specific. Local development might use user secrets, environment variables, or local configuration files. Production might use a cloud configuration service, secret manager, injected environment variables, or platform-level settings. The principle is the same as protecting connection strings in older systems, but the mechanisms are broader.

Security extends beyond application login. Authentication and authorization still matter, but so do service identities, secret rotation, container image scanning, dependency updates, cloud permissions, TLS, and deployment access.

Reliability is no longer just "keep the server up." It includes health checks, restart behavior, retries, timeouts, queue handling, horizontal scaling, database failover, and safe deployment strategies.

Deployment becomes a repeatable process. A production release should not depend on a person remembering which files to copy or which server setting to change. The goal is not automation for its own sake. The goal is reducing variation between releases.

Observability replaces server guessing. When an application runs in multiple instances, containers, or managed platforms, logging into the server is often not the first diagnostic move. You need structured logs, metrics, traces, health checks, alerts, and dashboards to understand what the system is doing.

Scaling moves from buying a bigger server to choosing the right boundary. Sometimes the answer is a larger database. Sometimes it is more application instances. Sometimes it is caching, background processing, or queue-based work. Sometimes the correct answer is to simplify the design.

Persistence remains central. SQL Server is still a major part of many .NET systems, including modern ones. The difference is that it may be hosted as a managed database, run in a container for local development, accessed through Entity Framework Core or Dapper, and deployed alongside migration automation.

Cost becomes a design concern. In a traditional environment, cost was often hidden behind infrastructure budgets. In cloud environments, architectural decisions can affect monthly bills directly. A chatty service, oversized database, unnecessary Kubernetes cluster, or inefficient AI call can have a visible cost.

The recurring production distinction is simple:

```
Local development optimizes for fast feedback.
Production optimizes for reliability, security, repeatability, and visibility.
```

Modern .NET gives you tools that can support both, but the design must be intentional.

Layer 6 — Tradeoffs and Alternatives

Modern does not always mean better. It means better aligned with current professional expectations and platform realities.

Use modern .NET practices when the application needs cross-platform hosting, cloud deployment, automated delivery, containerized environments, strong observability, frequent change, or integration with managed services.

Do not assume every application needs every modern tool. A small internal application may not need Kubernetes. A batch utility may not need a distributed architecture. A stable line-of-business system may benefit more from automated tests and clear deployment scripts than from a wholesale platform migration.

Simpler alternatives remain valid:

- IIS hosting can still be appropriate for some ASP.NET Core applications.
- A single well-designed application can be better than a premature set of microservices.
- A managed platform-as-a-service option can be simpler than running Kubernetes.
- SQL Server can remain the right database.
- Manual approval gates can be appropriate even inside automated pipelines.

More advanced alternatives exist for systems that need them:

- container orchestration;
- infrastructure as code;
- service meshes;
- event-driven architecture;
- distributed tracing;
- multi-region deployment;
- AI agents and retrieval-augmented generation.

The common overengineering mistake is treating the modern stack as a checklist. Docker, Kubernetes, Redis, queues, cloud platforms, and AI services each solve specific problems. They also add operational responsibility. A strong modern .NET developer knows both sides of that tradeoff.
The better question is not "Are we using the newest thing?" It is "Which system boundary is causing pain, and which tool makes that boundary clearer, safer, or easier to change?"

Layer 7 — Interview Perspective

Interviewers do not usually expect a returning .NET developer to be a deep expert in every modern platform tool. They do expect an accurate map.
Concepts commonly tested:

- the difference between .NET Framework, .NET Core, and modern .NET;
- why ASP.NET Core is not tied to IIS in the same way classic ASP.NET was;
- what containers solve and what they do not solve;
- why CI/CD exists;
- how cloud hosting changes deployment and operations;
- the difference between logs, metrics, and traces;
- why microservices are not automatically better than monoliths;
- how AI services fit into ordinary application architecture.

Representative questions:

- "What changed in .NET after .NET Framework?"
- "Why would a .NET team run an application on Linux?"
- "What problem does Docker solve for a .NET API?"
- "When would you choose a managed cloud service instead of a VM?"
- "What is the difference between deploying code and operating a production system?"
- "How would you modernize an older IIS-hosted application without rewriting everything?"

A strong answer connects concepts. For example:

> "Docker does not make the application scalable by itself. It packages the application and its runtime environment in a repeatable way. Scaling depends on where the container runs, whether the app is stateless, how configuration and secrets are handled, and what the database can support."

Common misconceptions:

- "Modern .NET means cloud only."
- "ASP.NET Core requires Linux."
- "Containers replace deployment pipelines."
- "Kubernetes is the default place to run every production app."
- "Microservices are the modern version of layered architecture."
- "AI integration means replacing normal application design."

Small design scenario:
You inherit an ASP.NET application hosted on Windows Server with IIS and SQL Server. Releases are manual, configuration differs between environments, and production issues are diagnosed by remote desktop access and log files on the server.
A good modernization discussion would not begin with "move it to Kubernetes." It would begin by identifying the current pain:

- Is deployment unreliable?
- Is the application hard to test?
- Are environments inconsistent?
- Is the server difficult to patch?
- Are production failures hard to diagnose?
- Is scaling actually required?

Then it would propose a sequence: get the code building consistently, add automated tests, clarify configuration, improve logging and health checks, choose a deployment target, and only then consider containers, cloud services, or orchestration if they solve a real problem.

Hands-On Lab

Objective:
Create a personal map of the modern .NET landscape and inspect the .NET installation on your machine.
Prerequisites:

- A development machine with the .NET SDK installed.
- A terminal: Windows Terminal, PowerShell, Command Prompt, macOS Terminal, or a Linux shell.
- No application code is required yet.

Steps:

1. Open a terminal.
2. Run the following command:

   ```
   dotnet --info
   ```

3. Identify the installed SDK version or versions.
4. Identify the installed runtime version or versions.
5. Note the operating system and architecture reported by the command.
6. Run:

```
dotnet --list-sdks
```

7. Run:

```
dotnet --list-runtimes
```

8. Draw a simple architecture map for a modern .NET application using these boxes:

```
Developer
Source control
Build and test pipeline
Application package or container image
Runtime host
Database
Observability
Users
```

9. Add arrows showing how code moves from development to production and how production feedback returns to the team.

Expected results:

- You can name the .NET SDKs and runtimes installed on your machine.
- You can explain the difference between local development, build automation, deployment, runtime hosting, and production feedback.
- You have a first-pass map of the modern .NET system this book will fill in.

Validation commands:

```
dotnet --info
dotnet --list-sdks
dotnet --list-runtimes
```

Troubleshooting notes:

- If `dotnet` is not recognized, the SDK may not be installed or may not be on your `PATH`.
- If only runtimes are installed, you may be able to run .NET applications but not build them.
- If several SDKs are installed, that is normal. Modern .NET versions can live side by side.

Knowledge Check

1. Why is “modern .NET” more than a newer runtime version?
2. In what ways is a container similar to a disciplined deployment package, and where does that analogy break down?
3. Why did the center of gravity move from individual servers toward delivery platforms?
4. What production concerns remain the same whether an application runs under IIS, in a container, or on a cloud platform?
5. Why is Kubernetes not the natural starting point for every modernization effort?
6. How does AI-assisted development differ from building AI-enabled application features?
7. Why should a returning .NET developer understand CI/CD even before becoming a deployment specialist?
8. What does observability provide that server access alone does not?

Summary

Modern .NET development begins with the same durable skills you already have: C#, HTTP, SQL, configuration, deployment judgment, and production discipline. The landscape around those skills has changed.

The old model placed the Windows server and IIS near the center. The modern model places the application inside a delivery platform made of source control, automated validation, repeatable packaging, cloud or container hosting, managed dependencies, security boundaries, and production telemetry.

.NET adapted to that world by becoming cross-platform, SDK-centered, cloud-friendly, container-friendly, and better integrated with modern configuration, logging, dependency injection, hosting, and AI patterns.

The rest of this book fills in the map one layer at a time. First you will learn the major technologies and workflow. Then you will build .NET applications. Then you will run them on Linux and in containers. Then you will deploy, observe, secure, scale, and reason about them as production systems.

The goal is not to chase novelty. The goal is to become fluent again in the professional environment where modern .NET systems are built and operated.

Sources

- Microsoft Learn, ".NET releases, patches, and support": https://learn.microsoft.com/en-us/dotnet/core/releases-and-support
- Microsoft, ".NET Support Policy": https://dotnet.microsoft.com/en-us/platform/support/policy
- Microsoft Learn, "Microsoft .NET and .NET Core - Microsoft Lifecycle": https://learn.microsoft.com/en-us/lifecycle/products/microsoft-net-and-net-core
- Microsoft Learn, ".NET container images": https://learn.microsoft.com/en-us/dotnet/core/docker/container-images
- Microsoft Learn, "Develop .NET apps with AI features": https://learn.microsoft.com/en-us/dotnet/ai/overview

Further Reading

- Microsoft Learn, "What's new in .NET 10": https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview
- Microsoft Learn, "Install .NET on Windows": https://learn.microsoft.com/en-us/dotnet/core/install/windows
- Microsoft Learn, "AI apps for .NET developers": https://learn.microsoft.com/en-us/dotnet/ai/

CHAPTER 2

The Modern Technology Stack

Chapter Purpose

Chapter 1 rebuilt the map. This chapter labels the major landmarks.

The modern .NET stack is not a single product. It is a working combination of runtime, web framework, operating system, packaging model, deployment platform, data stores, source-control system, delivery pipeline, and increasingly AI services.

If you come from a Windows, IIS, SQL Server, and Visual Studio background, the older stack may have felt vertically integrated. Microsoft supplied the language, runtime, IDE, web server integration, database, operating system, and deployment target. You could build a serious enterprise application while staying almost entirely inside one vendor ecosystem.

That world still exists, but the center widened.

Modern .NET remains a Microsoft-led ecosystem, but professional .NET work now commonly touches Linux, Docker, Kubernetes, Redis, GitHub, cloud platforms, open-source libraries, managed services, and AI providers. Some of these tools were adopted because they solved problems Microsoft products already faced. Some became industry standards outside the Microsoft ecosystem and .NET had to meet developers where the industry had moved. That is not a weakness. It is one of the reasons modern .NET is viable in cloud-native systems.

The purpose of this chapter is to introduce the major technologies without teaching their implementation details yet. You should finish this chapter able to explain what each technology is for, how it relates to the technologies around it, and where it fits in a complete application system.

This chapter deliberately stays at the map level. Later chapters teach the mechanics.

Where This Fits

Chapter 1 described the shift from server-centered development to platform-centered development. Chapter 2 turns that platform into named components.

A typical modern .NET application stack might look like this:

```
Users
  |
  v
Web or API edge
  |
  v
ASP.NET Core application
  |
  +-- SQL Server or managed relational database
  |
  +-- Redis cache
  |
  +-- Queue, background worker, or external service
  |
  +-- AI service when the application needs model behavior
```

```
  |
  v
Logs, metrics, traces, and alerts

Source code -> GitHub -> CI/CD -> package or container image -> host platform
```

The technologies in the roadmap do not all sit at the same layer:

- .NET is the runtime and developer platform.
- ASP.NET Core is the web and API framework.
- Linux is a common operating-system host.
- Docker is a packaging and local runtime tool for containers.
- Kubernetes is an orchestration platform for running containers at scale.
- SQL Server is the relational database many enterprise .NET systems already know.
- Redis is a fast in-memory data store often used for caching and coordination.
- Cloud platforms provide managed compute, databases, identity, networking, and operations.
- GitHub is commonly the source-control and collaboration hub.
- CI/CD is the automated path from code change to validated artifact and, eventually, deployment.
- AI services provide model-backed behavior that applications can call like other external dependencies.

This chapter introduces all of them together so later chapters can zoom in without forcing you to learn the system vocabulary from scratch.

Connection to the Reader's Existing Model

The easiest way to understand the modern stack is to compare each layer to something familiar.

.NET is still the platform you write C# against. The difference is that modern .NET is cross-platform, SDK-centered, open-source, and released on a regular cadence. It is less tied to a particular Windows installation than .NET Framework felt.

ASP.NET Core still handles HTTP requests, routing, application services, configuration, logging, and responses. The familiar IIS mental model helps, but ASP.NET Core is not just "classic ASP.NET with new project files." It can run behind IIS, behind a Linux reverse proxy, inside a container, or on a managed cloud platform.

Linux is the operating system you are most likely to encounter when .NET code runs outside Windows. Think of it as another production host with different filesystem, process, permission, service, and shell conventions. You do not need to become a Linux administrator to be productive, but you do need enough fluency to read paths, inspect processes, understand permissions, and interpret logs.

Docker is closest to a disciplined deployment package plus a lightweight process boundary. It is not a virtual machine in the traditional Hyper-V sense. It packages an application and its runtime dependencies as an image, then runs that image as an isolated process called a container.

Kubernetes is closest to a large-scale operations control plane. If a load-balanced IIS farm answers "which server receives the request?", Kubernetes also asks "which containers should exist, where should they run, how should they be replaced, how are they reached, and how is desired state maintained?" That analogy is useful, but Kubernetes is broader and more abstract than a web farm.

SQL Server is already familiar. The modern shift is not that relational databases disappeared. It is that SQL Server may run on Windows, Linux, in a container for development, in Azure SQL Database, in Azure SQL Managed Instance, or on a VM. The database remains a strong consistency and persistence boundary.

Redis is not a replacement for SQL Server. Think of it as a very fast in-memory structure server that can hold cached data, counters, distributed locks, session-like state, streams, or temporary coordination data. It solves a different class of problems from a relational database.

Cloud platforms are the modern equivalent of renting a data center where many infrastructure concerns are exposed as APIs and managed services. They replace some hardware and server administration with service selection, identity, networking, cost management, and operational design.

GitHub is more than a remote Git folder. In many teams it is the collaboration surface for pull requests, code review, security scanning, issue tracking, automation, and package publishing.

CI/CD replaces memory-based release discipline with executable release discipline. If you previously relied on a build machine, deployment checklist, and human care, CI/CD moves more of that process into source-controlled automation.

AI services are external intelligence dependencies. From an architecture point of view, they are closer to calling a payment processor, search service, or document service than to adding an ordinary class library. They introduce latency, cost, security, prompt design, evaluation, and correctness concerns.

Layer 1 — Conceptual Model

The modern technology stack can be understood as five cooperating layers:

```
Application layer
  .NET, ASP.NET Core, C#, application libraries

Data and dependency layer
  SQL Server, Redis, queues, files, APIs, AI services

Runtime and packaging layer
  Windows, Linux, Docker, container images

Platform and operations layer
  Cloud platforms, Kubernetes, identity, networking, observability

Delivery layer
  GitHub, pull requests, CI/CD, artifact registries, release automation
```

The application layer is where most business code lives. This is where you write APIs, validation, business rules, background services, and integrations.

The data and dependency layer is where the application stores durable state, reads fast temporary state, communicates with other systems, and calls external capabilities.

The runtime and packaging layer answers the question, “What exactly runs, and what does it require from the machine?” This includes the OS, installed runtime or container image, process environment, ports, files, certificates, and startup behavior.

The platform and operations layer answers the question, “Where does it run, how is it secured, how does it scale, and how do we know it is healthy?”

The delivery layer answers the question, “How does a code change become a validated production change?”

No single layer is the whole stack. A developer who only knows the application layer may write good C# but struggle to diagnose production. A developer who only knows platform tools may ship elaborate infrastructure for a simple problem. Modern .NET fluency means understanding how the layers cooperate.

Layer 2 — System Relationships

Each technology has inputs, outputs, dependencies, ownership boundaries, and failure modes.

.NET takes source code, project files, NuGet packages, SDKs, and configuration as inputs. It produces assemblies, applications, packages, and runtime behavior. The development team usually owns the code and package choices; the platform or operations team may own installed runtimes, base images, and runtime policies.

ASP.NET Core takes HTTP requests, configuration, dependency registrations, and middleware as inputs. It produces HTTP responses, logs, metrics, and calls to dependencies. It fails when routing is wrong, configuration is missing, dependencies are unavailable, startup fails, or runtime exceptions escape.

Linux takes processes, files, users, groups, sockets, services, and environment variables as operating-system concepts. A .NET developer usually does not own the whole Linux estate, but they may own enough of the application process to read logs, understand paths, inspect environment variables, and troubleshoot container behavior.

Docker takes a Dockerfile, application output, base images, and configuration as inputs. It produces container images and containers. The development team often owns the application image. A platform team may own the registry, base image policy, runtime host, and security scanning.

Kubernetes takes declarative resource definitions that describe desired state. It attempts to keep the actual cluster state aligned with those definitions. The application team may own deployment definitions for its service; a platform team often owns the cluster, networking, ingress, policies, secrets integration, and shared observability.

SQL Server takes schema, queries, transactions, indexes, data files, memory, storage, and connections. It produces durable state and query results. Its failure boundaries include locking, blocking, disk, memory, schema migration, backup, recovery, permissions, and network access.

Redis takes keys, values, data structures, expiration rules, memory, and client connections. It produces fast reads, writes, and coordination behavior. Its failure boundaries include memory pressure, eviction policy, persistence configuration, clustering, and the risk of treating cached data as if it were durable relational truth.

Cloud platforms take resource definitions, subscriptions or accounts, permissions, regions, networks, and billing rules. They produce hosted services. Their failure boundaries include identity, quota, cost, region availability, misconfiguration, provider limits, and shared-responsibility misunderstandings.

GitHub takes commits, branches, pull requests, issues, workflow files, and repository permissions. It produces review history, source history, automation events, and collaboration records. Its failure boundaries include branch-policy gaps, secret leaks, broken workflows, weak review practices, and permissions that are too broad.

CI/CD takes repository events, workflow definitions, build agents, secrets, tests, package feeds, and deployment credentials. It produces build results, test reports, packages, images, deployments, and audit trails. Its failure boundaries include flaky tests, missing environment parity, credential problems, incomplete rollback plans, and automation that hides rather than reduces risk.

AI services take prompts, input data, context, model choices, safety settings, tools, embeddings, and credentials. They produce generated text, structured outputs, classifications, embeddings, or tool calls. Their failure boundaries include latency, cost, hallucination, data leakage, model drift, evaluation gaps, and ambiguous ownership of generated behavior.

The relationships matter because production failures rarely respect chapter boundaries. A slow request might involve ASP.NET Core routing, Redis cache misses, SQL Server query plans, cloud network latency, and missing telemetry. The stack is a system.

Layer 3 — Core Mechanics

The smallest useful example is a web API with a database, cache, and delivery pipeline:

```
GitHub repository
  |
  v
CI builds and tests the .NET solution
  |
```

```
  v
Docker image is produced
  |
  v
Cloud platform runs the ASP.NET Core container
  |
  +-- SQL Server stores orders
  |
  +-- Redis caches product lookups
  |
  +-- AI service summarizes support notes
  |
  v
Observability records logs, metrics, and traces
```

The core terminology starts here.

A runtime executes compiled application code. In modern .NET, the runtime is not the same thing as the SDK. The SDK builds and publishes applications; the runtime runs them.

A framework provides reusable application structure. ASP.NET Core is the web framework used to build HTTP APIs, web applications, real-time apps, and services.

An operating system hosts processes, files, networking, permissions, and resource limits. Windows and Linux can both host modern .NET applications.

A container image is a packaged filesystem and metadata template used to create containers. A container is a running instance of that image.

An orchestrator manages many containers across machines. Kubernetes is the dominant open-source orchestrator and is widely used in cloud-native environments, but it is not required for every application.

A relational database stores structured durable data using tables, relationships, indexes, transactions, and queries. SQL Server remains a major enterprise relational database and, according to DB-Engines in July 2026, ranked third overall among database management systems.

An in-memory data store keeps data primarily in memory for speed. Redis is a prominent example and ranked eighth overall in the July 2026 DB-Engines ranking.

A cloud platform provides infrastructure and application capabilities as services: compute, databases, storage, networking, identity, observability, and AI.

A repository stores source history. Git is the version-control system; GitHub is a collaboration platform built around Git repositories and automation.

Continuous integration means automatically building and testing code changes after they are committed or proposed. Continuous delivery and deployment extend that automation toward release artifacts and environments.

An AI service exposes model behavior through an API or SDK. In modern .NET, Microsoft.Extensions.AI provides common abstractions for interacting with models and related AI capabilities across providers.

Layer 4 — Developer Workflow

Chapter 3 will teach the modern workflow in more detail. For this chapter, the workflow is about recognizing which tool participates at each stage.

Early in development, you mostly touch .NET, ASP.NET Core, SQL Server or a local database, maybe Redis, and GitHub. You run locally, test quickly, and commit changes.

As the application grows, Docker often enters the workflow to make the local environment more repeatable. Instead of every developer installing the same database, cache, and supporting services by hand, a team can describe a local multi-container environment.

When the team starts sharing changes, CI becomes important. A pull request should not depend only on "it works on my machine." It should trigger a build and tests in a clean environment.
When the team starts deploying repeatedly, CD becomes important. The question shifts from "Can we build it?" to "Can we safely move this exact validated artifact into an environment?"
As production needs grow, cloud services and observability become part of the daily workflow. Developers need to know how configuration reaches the application, where logs go, how database access is secured, how health is reported, and how deployments can be rolled back.
Kubernetes usually appears after these basics. It is most useful when a team needs a consistent platform for many containerized workloads, not when it is trying to learn containers for the first time.
AI services may appear in two places. AI coding assistants may help during development. AI application services may become runtime dependencies when the product needs summarization, classification, chat, extraction, embeddings, or agentic workflows.
Useful orientation commands:

```
dotnet --info
git --version
docker --version
kubectl version --client
```

On Windows PowerShell, the commands are the same:

```
dotnet --info
git --version
docker --version
kubectl version --client
```

These commands are not a setup requirement for this chapter. They simply show which parts of the stack are already present on your machine.

Layer 5 — Production Usage

In production, the technology stack becomes a set of responsibility boundaries.
.NET and ASP.NET Core define the application runtime boundary. Production questions include which .NET version is supported, whether the app is patched, how configuration is loaded, how logs are emitted, how health is exposed, and how dependency failures are handled.
Linux or Windows defines the host operating-system boundary. Production questions include patching, file permissions, service identity, certificates, network access, process limits, and diagnostic access.
Docker defines the packaging boundary. Production questions include base-image trust, image size, vulnerability scanning, registry access, immutable tags, startup behavior, and whether runtime configuration is separated from the image.
Kubernetes defines the orchestration boundary. Production questions include replicas, health probes, rolling updates, secrets integration, network policy, resource limits, storage, cluster ownership, and operational complexity.
SQL Server defines the durable data boundary. Production questions include backup and restore, high availability, transaction isolation, migration strategy, index health, query performance, permissions, and cost.
Redis defines the fast state boundary. Production questions include whether the data is disposable or durable, how memory is sized, how eviction works, whether clustering is needed, and how cache misses affect the database.
Cloud platforms define the managed-service boundary. Production questions include region choice, identity, networking, quotas, compliance, platform service-level agreements, cost controls, and shared responsibility.
GitHub and CI/CD define the delivery boundary. Production questions include who can approve changes, which checks are required, how secrets are protected, how artifacts are versioned, and how deployments are audited.

AI services define the model boundary. Production questions include data privacy, prompt injection, output validation, model selection, cost per call, evaluation, latency, fallback behavior, and observability of AI decisions.

The state-of-the-art direction across these areas is not "use more tools." It is platform engineering: make the correct path easy for teams. A mature organization gives developers paved paths for common tasks such as creating an API, adding a database, creating a pipeline, publishing a container, connecting to secrets, emitting telemetry, and deploying safely.

That is the modern production goal: not heroic deployment knowledge, but repeatable systems that make the safe thing normal.

Layer 6 — Tradeoffs and Alternatives

The modern stack is full of good tools, and good tools can still be wrong for the job.

.NET competes most often with ecosystems such as Java, Go, Node.js, Python, Rust, and Kotlin. .NET is strong for enterprise applications, APIs, cloud services, Windows integration, high-performance server workloads, and teams that value C# and mature tooling. It may not be the natural first choice when a team is already deeply invested in another ecosystem or when a specialized runtime dominates a domain.

ASP.NET Core competes with frameworks such as Spring Boot, Express, FastAPI, Django, Rails, Laravel, and Go HTTP frameworks. ASP.NET Core is strong for typed, high-performance APIs and enterprise web systems. It can be excessive for a tiny script-like endpoint, and it may be unfamiliar to teams centered on JavaScript or Python.

Linux competes with Windows Server as a host for .NET workloads. Linux is common for containers and cloud-native hosting. Windows remains appropriate for applications with Windows-specific dependencies, legacy COM integration, classic .NET Framework, or operational teams built around Windows tooling.

Docker competes with direct host installation, virtual machines, language- specific packaging, and platform-native deployment packages. Docker is useful when environment repeatability matters. It can be unnecessary for a simple internal utility or a platform-as-a-service deployment that already abstracts packaging well.

Kubernetes competes with simpler options: Azure App Service, Azure Container Apps, AWS Elastic Beanstalk, AWS ECS, Google Cloud Run, ordinary VMs, and managed platform services. Kubernetes is powerful when many services need a shared orchestration model. It is often overengineering for one or two modest applications.

SQL Server competes with PostgreSQL, MySQL, Oracle, SQLite, cloud-native managed databases, document databases, and analytical stores. SQL Server is a strong choice for enterprise relational workloads, T-SQL expertise, Microsoft tooling, and existing estates. PostgreSQL is a major open-source alternative; SQLite is excellent for embedded and local scenarios; document stores fit different data shapes.

Redis competes with in-process memory caches, Memcached, database caching, message brokers, and specialized streaming systems. Redis is useful when fast shared state or data structures matter. It is not a substitute for relational integrity, durable event logs, or careful database design.

Cloud platforms compete with on-premises hosting, colocation, private cloud, and hybrid designs. Azure is often natural for Microsoft-centered enterprises, AWS is broad and deeply established, and Google Cloud has strengths in data, analytics, Kubernetes heritage, and AI. The right choice is usually shaped by organizational skills, contracts, compliance, existing workloads, and service fit.

GitHub competes with Azure DevOps, GitLab, Bitbucket, and self-hosted Git systems. GitHub is widely used for open source and increasingly common in enterprise collaboration. Azure DevOps remains common in Microsoft enterprise shops with established Boards, Repos, and Pipelines usage.

CI/CD tools include GitHub Actions, Azure Pipelines, GitLab CI, Jenkins, TeamCity, CircleCI, Buildkite, and others. The best tool is the one your team can operate reliably with clear secrets management, repeatable builds, and useful feedback.

AI services compete across providers and hosting models: OpenAI, Azure OpenAI, Azure AI Foundry, GitHub Models, Amazon Bedrock, Google Gemini, local models through tools such as Ollama, and traditional ML through

ML.NET. Modern .NET's direction is to provide abstractions that let applications use model services without binding every part of the code to one provider.
The most common overengineering mistake is stacking all of these choices into the first version of an application. A strong architecture grows the stack when the system has earned the complexity.

Layer 7 — Interview Perspective

Interviewers use stack questions to test whether you understand boundaries, not whether you can recite logos.
Concepts commonly tested:

- the role of .NET versus ASP.NET Core;
- why Linux matters to modern .NET;
- the difference between containers and virtual machines;
- what Kubernetes adds on top of containers;
- why SQL Server and Redis are complementary rather than interchangeable;
- how GitHub, pull requests, and CI relate to team quality;
- what cloud platforms manage and what the team still owns;
- how AI services fit into an ordinary application architecture.

Representative questions:

- "Where does ASP.NET Core end and the hosting platform begin?"
- "Why would a .NET team use Docker?"
- "When would you avoid Kubernetes?"
- "What belongs in SQL Server versus Redis?"
- "How is GitHub Actions different from Git itself?"
- "What does a cloud provider manage for you, and what remains your responsibility?"
- "How would you add an AI feature without making the whole application depend on one model provider?"

A strong answer names the boundary and the tradeoff.
For example:

> "Redis can reduce load on SQL Server by caching frequently read data, but SQL Server remains the system of record. If Redis is unavailable, the application should either fall back to SQL Server or fail in a controlled way, depending on the feature."

Common misconceptions:

- ".NET is only a Windows platform."
- "ASP.NET Core requires IIS."
- "Docker is the same as Kubernetes."
- "Kubernetes makes applications cloud-native automatically."
- "Redis is just a faster database."
- "CI/CD is only a deployment script."
- "Cloud means no operations work."
- "AI services can be treated like deterministic libraries."

Small design scenario:
You are asked to design the first modern version of an internal order-tracking API. The company knows C#, SQL Server, and Windows Server. It wants to become more modern but does not yet have a large platform team.
A sensible stack might be:

- .NET and ASP.NET Core for the API;
- SQL Server for orders and durable business data;
- GitHub or Azure DevOps for source control and pull requests;
- CI to build and test every change;
- Docker for repeatable local dependencies and possibly packaging;
- a managed cloud app service or container app for hosting;
- basic logging, health checks, and metrics;
- Redis only if caching becomes necessary;
- Kubernetes only if the organization later needs orchestration at scale;
- AI services only when there is a concrete product feature, such as support note summarization or order-message classification.

The strong answer is not maximal. It is staged.

Hands-On Lab

Objective:
Create a stack map for a modern .NET application and classify each technology by responsibility.
Prerequisites:

- Chapter 1 completed.
- A text editor or notebook.
- Optional local installations of .NET, Git, Docker, and Kubernetes CLI tools.

Steps:

1. Draw five columns:

   ```
   Application | Data | Packaging/Runtime | Platform/Ops | Delivery
   ```

2. Place each technology from this chapter into the most natural column:

   ```
   .NET
   ASP.NET Core
   Linux
   Docker
   Kubernetes
   SQL Server
   Redis
   Cloud platform
   GitHub
   CI/CD
   AI services
   ```

3. For each technology, write one sentence explaining what problem it solves.
4. For each technology, write one sentence explaining what it does not solve.
5. Mark which technologies are essential for a first ASP.NET Core API.
6. Mark which technologies are optional until the system grows.
7. If you have the tools installed, run:

   ```
   dotnet --info
   git --version
   docker --version
   kubectl version --client
   ```

8. Rewrite the stack as a simple architecture diagram for an order-tracking API.

Expected results:

- You can describe the role of each major technology.
- You can separate application concerns from data, runtime, platform, and delivery concerns.
- You can identify which technologies are foundational and which are situational.

Validation commands:

```
dotnet --info
git --version
docker --version
kubectl version --client
```

Troubleshooting notes:

- If `docker` is not installed, complete the lab conceptually. Docker is taught later.
- If `kubectl` is not installed, that is fine. Kubernetes is intentionally not required yet.
- If you cannot decide where a technology belongs, ask what boundary it owns: code, data, runtime, platform, or delivery.

Knowledge Check

1. Why is ASP.NET Core not the same layer as Docker or Kubernetes?
2. What problem does Linux solve for modern .NET teams if Windows Server still exists?
3. Why is Docker useful even before an application runs in Kubernetes?
4. What does Kubernetes add beyond running a single container?
5. Why should SQL Server usually remain the system of record when Redis is added?
6. What is the difference between Git and GitHub?
7. Why is CI/CD part of the technology stack rather than only a team process?
8. What does a cloud platform manage, and what does it not manage for your application team?
9. Why should AI services be treated as runtime dependencies rather than ordinary deterministic libraries?
10. Which technologies in this chapter would you choose for the first version of a small internal API, and which would you postpone?

Summary

The modern .NET technology stack is a layered system.

.NET and ASP.NET Core provide the application foundation. SQL Server and Redis serve different data needs: durable relational state and fast shared in-memory state. Linux and Docker shape how applications run and how their environments are packaged. Kubernetes manages containerized workloads when scale and platform consistency justify its complexity. Cloud platforms provide managed compute, data, identity, networking, and operations services. GitHub and CI/CD turn source changes into validated, reviewable, repeatable delivery. AI services add model-backed behavior when the product needs capabilities that ordinary deterministic code does not provide.

The stack is not a checklist. It is a set of choices around boundaries. The professional skill is knowing which boundary you are addressing and whether the tool adds more clarity than complexity.

The next chapter moves from the technology map to the day-to-day workflow: how modern teams create, review, test, and move .NET changes through the software lifecycle.

Sources

- Microsoft Learn, ".NET releases, patches, and support": https://learn.microsoft.com/en-us/dotnet/core/releases-and-support
- Microsoft Learn, "ASP.NET documentation": https://learn.microsoft.com/en-us/aspnet/core/
- Docker Docs, "What is Docker?": https://docs.docker.com/get-started/docker-overview/
- Kubernetes Documentation, "Concepts": https://kubernetes.io/docs/concepts/
- Microsoft Learn, "What is SQL Server?": https://learn.microsoft.com/en-us/sql/sql-server/what-is-sql-server
- Redis, "What is Redis?: An Overview": https://redis.io/tutorials/what-is-redis/
- GitHub Docs, "GitHub Actions documentation": https://docs.github.com/en/actions
- Microsoft Learn, "Use continuous integration": https://learn.microsoft.com/en-us/devops/develop/what-is-continuous-integration
- Microsoft Azure, "What is cloud computing?": https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-cloud-computing/
- Microsoft Learn, "Develop .NET apps with AI features": https://learn.microsoft.com/en-us/dotnet/ai/overview
- DB-Engines, "DB-Engines Ranking": https://db-engines.com/en/ranking/
- CNCF, "2025 Annual Cloud Native Survey" announcement: https://www.cncf.io/announcements/2026/01/20/kubernetes-established-as-the-de-facto-operating-system-for-ai-as-production-use-hits-82-in-2025-cncf-annual-cloud-native-survey/
- Stack Overflow, "2025 Developer Survey: Technology": https://survey.stackoverflow.co/2025/technology/

Further Reading

- Microsoft Learn, ".NET + AI ecosystem tools and SDKs": https://learn.microsoft.com/en-us/dotnet/ai/dotnet-ai-ecosystem
- Google Cloud, "Google Cloud overview": https://docs.cloud.google.com/docs/overview
- AWS, "What is cloud computing?": https://docs.aws.amazon.com/whitepapers/latest/aws-overview/what-is-cloud-computing.html
- Docker Docs, "What is a container?": https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

CHAPTER 3

Modern Development Workflow

Chapter Purpose

The previous chapter named the major technologies in the modern .NET stack. This chapter shows how work moves through that stack during an ordinary professional development cycle.

If your baseline is Visual Studio 2019, Team Foundation Server or an older Git setup, manual QA handoffs, and deployments coordinated through people and servers, the modern workflow can look busier than the old one. There are pull requests, automated checks, cloud development environments, code review tools, pipeline statuses, package feeds, container images, security scans, and AI coding assistants.

The purpose of all that machinery is not ceremony. It is controlled feedback.

Modern development workflow exists because professional software teams learned the cost of late integration, unclear review, environment drift, manual deployment, and production surprises. The newer workflow tries to make each change small enough to understand, visible enough to review, automated enough to validate, and traceable enough to operate.

This chapter does not teach full CI/CD implementation. Chapter 8 covers automated testing and CI basics, and Chapter 16 covers deployment pipelines. Here, the goal is to understand the complete lifecycle at a high level: local development, source control, pull requests, testing, code reviews, AI coding assistants, development environments, and how code moves toward production.

Where This Fits

Chapter 1 shifted the mental model from server-centered development to platform-centered development. Chapter 2 identified the technologies in that platform. Chapter 3 connects them through time.

A modern workflow is a loop:

```
Plan a small change
        |
        v
Create or update code locally
        |
        v
Run local build and tests
        |
        v
Commit to a short-lived branch
        |
        v
Open a pull request
        |
        v
Review, discuss, and revise
        |
        v
```

```
Automated checks validate the change
        |
        v
Merge to the main branch
        |
        v
Build artifact or package
        |
        v
Deploy through a controlled process
        |
        v
Observe production behavior
        |
        v
Feed learning back into the next change
```

The key idea is that development does not end when code compiles. In a modern team, the workflow connects development, delivery, and operations. Microsoft describes DevOps as uniting people, process, and technology across planning, development, delivery, and operations. That is the lifecycle this chapter is mapping.
This workflow is now common across ecosystems, not only .NET. Java, Node.js, Python, Go, and .NET teams all use variants of this pattern because it solves a team coordination problem, not a language-specific problem.

Connection to the Reader's Existing Model

The familiar workflow probably had these landmarks:

- open the solution in Visual Studio;
- make a change;
- run locally with IIS Express or a local IIS site;
- maybe run unit tests;
- check in code;
- wait for a build or manually produce one;
- hand off to QA or operations;
- deploy by copying files, running a script, or using a release tool;
- troubleshoot on the server if something failed.

Modern development keeps the same broad intent but makes more of the workflow explicit and repeatable.
Visual Studio is still a powerful local environment, but it is no longer the only place where the project must build. A build should work from the command line, on another developer's machine, and on a clean build agent.
Source control is no longer just a backup and history mechanism. It is the coordination point for branches, pull requests, review, automated checks, security scanning, release notes, and sometimes infrastructure definitions.
A pull request is similar to a formal code review or change request, but it is attached directly to the code diff, tests, comments, approvals, and automation results.
Automated tests are similar to regression test scripts, but they execute as part of the normal change process instead of waiting for a separate late-stage test phase.
A CI check is similar to a build-server validation, but modern teams expect it to run frequently, consistently, and close to the code review.
A development environment is still a workstation-like place where you write and run code. The difference is that the environment may be local, container based, cloud hosted, or standardized through repository configuration.

An AI coding assistant is not a replacement for the developer. It is closer to an always-available pair programmer that can suggest code, explain APIs, draft tests, summarize changes, and help navigate unfamiliar areas. The analogy is useful as long as you remember that the assistant does not own correctness, security, or design judgment.
The old model often relied on expert memory: the senior developer knew how to build it, the release engineer knew how to deploy it, and the production admin knew which server setting mattered. The modern model tries to turn that memory into visible workflow.

Layer 1 — Conceptual Model

Modern development workflow is the repeatable path by which a team turns an idea into a safe, reviewed, validated, deployable software change.
It exists to solve five recurring problems.
First, integration delay. When developers work separately for too long, changes conflict, assumptions diverge, and bugs are discovered late. Short-lived branches, frequent commits, pull requests, and CI reduce the cost of bringing work together.
Second, environment drift. If every developer's machine, test server, and production server is slightly different, success in one place does not predict success in another. SDK-based builds, containerized dependencies, documented setup, and cloud development environments reduce drift.
Third, review ambiguity. A hallway conversation or email thread is easy to lose. Pull requests connect the code, conversation, tests, and approval record.
Fourth, manual validation. Humans are good at judgment and design review. Humans are poor at repeatedly remembering every mechanical check. Automated builds, tests, linters, formatters, and scans handle repeatable validation so people can focus on meaning.
Fifth, production disconnect. In older workflows, developers could finish a change and lose sight of it once it crossed into QA or operations. Modern workflow keeps feedback connected through deployment status, telemetry, incidents, and post-release learning.
What the workflow does not solve by itself:

- It does not guarantee good architecture.
- It does not make tests valuable just because they are automated.
- It does not replace code review judgment.
- It does not remove the need for production ownership.
- It does not make AI-generated code correct.
- It does not make a broken team healthy through tools alone.

The workflow is a support system for disciplined engineering. It makes good habits easier to repeat.

Layer 2 — System Relationships

The modern workflow has several connected actors and boundaries.
The developer owns the local change. Inputs include a work item, branch, existing code, local environment, tests, and context from previous chapters or documentation. Outputs include commits, tests, documentation updates, and a pull request.
The repository owns shared history. It receives commits and branches. It outputs version history, diffs, merge records, tags, and sometimes release artifacts. Its failure boundaries include bad branching practices, weak permissions, missing history, and unclear ownership.
The pull request owns review coordination. It receives a proposed diff, description, linked work item, review comments, automated check results, and updates from the author. It outputs an approved, rejected, or revised

change. Its failure boundaries include oversized changes, vague descriptions, rubber-stamp review, stale branches, and unresolved discussion.

The test suite owns repeatable validation. It receives compiled code, test data, dependencies, and environment configuration. It outputs pass/fail signals and diagnostics. Its failure boundaries include flaky tests, missing coverage for important behavior, over-mocked tests, slow feedback, and tests that assert implementation details instead of behavior.

The CI system owns clean validation. It receives repository events such as commits or pull requests. It checks out the code, restores dependencies, builds the solution, runs tests, and reports status. Its failure boundaries include broken agents, missing secrets, unavailable package feeds, nondeterministic tests, and mismatched SDK versions.

The code review process owns human judgment. It receives the author's intent, the diff, test evidence, design context, and automated results. It outputs feedback, suggestions, approval, or a request for changes. Its failure boundaries include reviewing style while missing behavior, deferring too much to automation, or treating review as personal criticism instead of shared ownership.

The development environment owns local productivity. It receives repository configuration, SDKs, tools, containers, secrets for local development, and IDE settings. It outputs fast feedback. Its failure boundaries include slow setup, hidden machine-specific assumptions, missing dependencies, and local-only success.

The AI coding assistant owns suggestion and acceleration, not authority. It receives prompts, visible code context, chat history, repository files, and possibly tool access depending on the environment. It outputs suggestions, explanations, edits, tests, or summaries. Its failure boundaries include plausible but wrong code, outdated assumptions, insecure suggestions, data exposure, and over-trust.

The deployment process owns movement toward runtime environments. In this chapter, deployment remains a high-level concept. Later chapters teach the mechanics. For now, understand that the output of development should be a validated change that can become a deployable artifact through a controlled path.

Layer 3 — Core Mechanics

The smallest useful workflow is a branch and pull request with automated validation.

```
main branch
  |
  +-- feature branch
        |
        v
      commits
        |
        v
      pull request
        |
        +-- human review
        |
        +-- automated build and tests
        |
        v
      merge
        |
        v
main branch with validated change
```

The main branch represents the shared integration line. Some teams call it `main`, some use `master`, and some use trunk-based development language. The name matters less than the rule: the shared line should be kept healthy.

A branch is an isolated line of work. Modern teams generally prefer short-lived branches because they reduce integration delay.
A commit is a recorded change. Good commits make the history understandable. They are not just save points; they are communication.
A pull request proposes that branch changes be merged into another branch. It contains the diff, discussion, review state, and automated checks.
A code review is a human evaluation of the proposed change. GitHub pull request reviews support comments, suggestions, approval, and requests for changes.
A check is an automated result attached to the proposed change. Checks might build the solution, run tests, enforce formatting, scan dependencies, or produce preview artifacts.
Continuous integration is the practice of automatically building and testing code when changes are committed to version control. It exists to catch integration problems early.
Continuous delivery is the broader practice of automating build, test, configuration, and deployment movement toward environments. This chapter mentions it only as part of the lifecycle. The detailed release pipeline comes later.
A development environment is the place where the developer writes and runs the application. It can be a local machine, a local container setup, a cloud environment such as GitHub Codespaces, or a managed developer workstation such as Microsoft Dev Box.
An AI coding assistant is a tool that uses large language models to assist with coding tasks. Examples include inline suggestions, chat-based code explanation, refactoring help, test generation, and debugging support. The state-of-the-art direction in 2026 is broader than autocomplete: assistants are increasingly integrated into IDEs, repositories, pull requests, terminals, cloud consoles, and agent-like workflows. That power makes review and judgment more important, not less.

Layer 4 — Developer Workflow

Here is the modern workflow in practical terms.
Start with a small piece of work. It might come from a backlog item, bug report, incident follow-up, architecture decision, or direct user need. The unit of work should be small enough that someone else can review it without reconstructing the entire system.
Update your local repository:

```
git status
git pull
```

Create a branch:

```
git switch -c feature/add-order-status
```

Run the application or tests before changing code when practical. This gives you a baseline:

```
dotnet build
dotnet test
```

Make the change in your editor or IDE. Visual Studio, Visual Studio Code, JetBrains Rider, Codespaces, and Dev Box can all participate in modern .NET workflows. The specific tool matters less than whether the workflow is repeatable outside one tool.
Use an AI coding assistant carefully if available. Good uses include asking it to explain unfamiliar code, draft a test shape, identify edge cases, summarize a diff, or suggest a refactoring. Risky uses include accepting large generated changes without understanding them, pasting secrets into prompts, or treating generated code as reviewed code.
Run local validation:

```
dotnet format --verify-no-changes
dotnet build
dotnet test
```

The exact commands vary by repository. Some teams use scripts, `make`, `just`, PowerShell, npm, Docker Compose, or custom build tools. The principle is that the repository should expose a known way to validate the change.
Commit the change:

```
git status
git add .
git commit -m "Add order status validation"
```

Push the branch:

```
git push -u origin feature/add-order-status
```

Open a pull request. The pull request description should explain what changed, why it changed, how it was tested, and any risk or follow-up work. If the change has screenshots, logs, migration notes, or API examples, include them.
Respond to review. Review is not a ceremony after the real work; it is part of the work. A reviewer may find a missed case, unclear name, fragile test, hidden security issue, or simpler design.
Wait for automated checks. If checks fail, fix the branch rather than hoping the failure is unrelated. If the failure is flaky, treat that as a workflow problem worth improving.
Merge after review and checks pass. Depending on team policy, the merge may create a merge commit, squash commits, or rebase. That policy is less important than keeping shared history understandable and the main branch healthy.
After merge, the change moves toward deployment. In a mature environment, the same change can be traced from work item to pull request, commit, build artifact, deployment, and production telemetry.

Layer 5 — Production Usage

Workflow choices affect production quality.
Configuration should be separated from code. Local development may use local settings or user secrets. CI may use test configuration. Production may use environment variables, managed secret stores, or platform configuration. Workflow should make it hard to accidentally commit secrets.
Security should shift earlier without pretending development can catch everything. Modern workflows often include dependency scanning, secret scanning, branch protection, least-privilege repository access, required reviews, and controlled deployment credentials. These practices reduce risk before the application reaches production.
Reliability starts before deployment. Small changes are easier to review, test, deploy, and roll back. Automated tests reduce regression risk. Pull requests capture design judgment. CI confirms that the change works outside the author's machine.
Deployment should consume validated artifacts. A common failure in older systems was rebuilding or modifying files during release in ways that made it unclear what actually reached production. Modern workflows try to build once, identify the artifact, and move that artifact through environments.
Observability closes the loop. Production logs, metrics, traces, errors, and alerts tell the team whether the change behaved as expected. Without feedback, deployment is a cliff.
Scaling affects workflow because larger teams need clearer boundaries. A three-person team can coordinate with conversation. A thirty-person team needs branch policies, ownership rules, review expectations, consistent local setup, and automation that prevents accidental damage.

Persistence affects workflow because database changes must be coordinated with application changes. A schema migration, data correction, or index change is not just "code." It has deployment ordering, rollback, locking, backup, and performance implications.

Cost affects workflow when builds, cloud development environments, preview environments, AI tools, and test infrastructure are metered. A modern workflow should give fast feedback without silently creating waste.

Local-development arrangements optimize for fast feedback. Production designs optimize for reliability, security, auditability, and recoverability. A good workflow keeps those goals connected without pretending they are identical.

Layer 6 — Tradeoffs and Alternatives

There is no single correct workflow for every team.

Small teams may use lightweight GitHub repositories, short branches, pull requests, and a few required checks. Larger enterprises may add issue tracking, change approval, security gates, compliance evidence, release trains, and environment promotion.

GitHub is widely used, especially for open source and many modern product teams. Azure DevOps remains common in Microsoft-oriented enterprises. GitLab, Bitbucket, Jenkins, TeamCity, CircleCI, and Buildkite are also valid choices. The workflow principles matter more than the brand: versioned code, review, automated validation, traceable delivery, and feedback from production.

Pull requests are not the only collaboration model. Some high-performing teams use trunk-based development with very short-lived branches and strong automation. Some regulated environments require heavier approval processes. Some open-source projects rely on maintainer review across forks. The best choice depends on team size, risk, deployment frequency, and compliance needs.

Local development is not always enough. A standard workstation may be fastest for many developers. A containerized setup may reduce dependency drift. GitHub Codespaces can provide cloud-hosted, repository-configured development environments. Microsoft Dev Box can provide secure, managed cloud development workstations. These options solve setup and consistency problems, but they add cost and administrative decisions.

AI coding assistants can accelerate exploration, routine code, test drafts, documentation, and unfamiliar API usage. They can also produce confident mistakes. Use them as collaborators under review, not as authorities.

Common overengineering mistakes:

- requiring a pull request for every tiny experimental spike;
- creating so many pipeline gates that developers stop trusting the system;
- running slow checks on every keystroke-level change;
- adding cloud development environments before local setup is understood;
- treating code review as permission bureaucracy instead of design feedback;
- accepting AI-generated code because it looks polished;
- confusing a sophisticated workflow with a healthy engineering culture.

Simpler alternatives are sometimes better. A small internal tool may need Git, a README, a build command, and a modest test suite before it needs preview environments and advanced deployment rings. More advanced alternatives are worthwhile when the risk and scale justify them: trunk-based development, ephemeral environments, progressive delivery, feature flags, policy-as-code, supply-chain signing, and platform-engineered paved paths.

The state-of-the-art direction is not heavier process. It is shorter feedback loops with stronger guardrails.

Layer 7 — Interview Perspective

Interviewers use workflow questions to learn how you work with a team.

Concepts commonly tested:

- the purpose of source control beyond backup;
- branch and pull request workflow;
- what belongs in a pull request description;
- the difference between local tests and CI checks;
- the purpose of code review;
- how to handle failed builds;
- how AI coding assistants should and should not be used;
- how a code change moves toward production.

Representative questions:

- “Walk me through your development workflow from task to merge.”
- “What makes a pull request easy to review?”
- “What should happen before code is merged to the main branch?”
- “How do you respond when CI fails but the code works locally?”
- “What is the difference between continuous integration and continuous delivery?”
- “How do you use AI coding tools responsibly?”
- “How would you improve a team that deploys manually and often has release surprises?”

A strong answer describes feedback loops. For example:

> “I try to keep changes small, run local validation before opening the pull request, explain the intent and test evidence, respond to review, and treat CI failures as real until understood. After merge, the change should be traceable through build and deployment so production feedback can be tied back to the code.”

Common misconceptions:

- “Source control is mainly a backup.”
- “A pull request is only for catching syntax mistakes.”
- “If it works locally, CI is optional.”
- “Code review is less important when tests pass.”
- “AI-generated code does not need the same review as human-written code.”
- “DevOps means developers do all operations work.”
- “Continuous delivery means every commit must go directly to production.”

Small design scenario:
You join a team maintaining a .NET line-of-business application. Developers work directly on a shared branch, releases are manual, tests are mostly run by QA near the end, and production fixes often happen under pressure.
A good modernization plan would be incremental:

- move to short-lived branches and pull requests;
- require a clear description and reviewer before merge;
- add a clean build check;
- add a small but meaningful automated test set;
- document local setup;
- protect the main branch;
- make release artifacts identifiable;
- improve production logging before attempting aggressive deployment automation;
- introduce AI coding tools with review and security expectations.

The answer should improve feedback before adding ceremony.

Hands-On Lab

Objective:
Practice the shape of a modern workflow using a small repository-level change.
Prerequisites:

- Git installed.
- .NET SDK installed.
- A local repository.
- No remote repository is required for the local parts of this lab.

Steps:

1. Inspect the repository state:

   ```
   git status
   ```

2. Create a branch:

   ```
   git switch -c workflow-practice
   ```

3. Inspect the installed .NET SDK:

   ```
   dotnet --info
   ```

4. If the repository has a solution or project, run:

   ```
   dotnet build
   dotnet test
   ```

   If the repository does not yet contain buildable code, write down the commands you expect the repository to support once code exists.

5. Create a short pull request draft in a text file or notebook with these headings:

   ```
   Summary
   Why
   Testing
   Risks
   Follow-up
   ```

6. Write a sample review comment against your own hypothetical change. Make it specific, respectful, and actionable.
7. Write one AI-assistant prompt that would be useful and safe for this work. For example:

   ```
   Review this small diff for edge cases and missing tests. Do not rewrite the
   implementation unless you find a specific issue.
   ```

8. Return to your original branch when finished:

   ```
   git switch -
   ```

Expected results:

- You can describe the workflow from branch to pull request to validation.
- You can distinguish local validation from CI validation.
- You can write a useful pull request description.
- You can state how AI assistance fits into review rather than replacing it.

Validation commands:

```
git status
git branch --show-current
dotnet --info
dotnet build
dotnet test
```

Troubleshooting notes:

- If `git switch -c` fails because the branch already exists, choose another branch name.
- If `dotnet build` fails because there is no project file, that is acceptable for this chapter. The next chapters begin building code.
- If `dotnet test` reports no test projects, note that as a future workflow gap rather than an error.
- If you use an AI assistant, do not paste secrets, private credentials, or confidential production data into the prompt.

Knowledge Check

1. Why is modern development workflow best understood as a feedback loop?
2. What problems do short-lived branches reduce?
3. What information should a pull request description provide to reviewers?
4. Why are automated checks useful even when reviewers are experienced?
5. How is continuous integration different from continuous delivery?
6. Why is “works on my machine” weaker evidence than a passing CI build?
7. What kinds of comments make code review more useful?
8. How can cloud development environments reduce onboarding friction?
9. What risks do AI coding assistants introduce into the workflow?
10. Why should production telemetry feed back into development planning?

Summary

Modern .NET development workflow is the path from idea to validated change to production feedback.
The core practices are not mysterious: work in small increments, keep source history clear, review proposed changes, automate repeatable validation, protect the shared branch, package changes consistently, and learn from production behavior. The tools have changed, but the goal is familiar: make software change safer.
Compared with older workstation-and-server workflows, the modern workflow places more responsibility in shared systems: Git repositories, pull requests, CI checks, documented development environments, controlled delivery, and observability. AI coding assistants now participate in that workflow, but they do not replace the developer’s responsibility for correctness, security, and design.
The next chapter begins the practical .NET layer. With the workflow map in place, you can now look at modern .NET versions, SDKs, project structure, cross-platform development, and language improvements as parts of a larger professional system.

Sources

- Microsoft Learn, “What is DevOps?”: https://learn.microsoft.com/en-us/devops/what-is-devops
- Microsoft Learn, “Use continuous integration”: https://learn.microsoft.com/en-us/devops/develop/what-is-continuous-integration

- Microsoft Learn, “What is continuous delivery?”: https://learn.microsoft.com/en-us/devops/deliver/what-is-continuous-delivery
- GitHub Docs, “Quickstart for reviewing pull requests”: https://docs.github.com/en/pull-requests/get-started/reviewing-pull-requests-quickstart
- GitHub Docs, “Requesting a pull request review”: https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/requesting-a-pull-request-review
- GitHub Docs, “What are GitHub Codespaces?”: https://docs.github.com/en/codespaces/about-codespaces/what-are-codespaces
- Microsoft Learn, “Open a dev box in VS Code”: https://learn.microsoft.com/en-us/azure/dev-box/how-to-set-up-dev-tunnels
- Microsoft Learn, “GitHub Copilot Fundamentals”: https://learn.microsoft.com/en-us/training/paths/copilot/

Further Reading

- Microsoft Learn, “Deliver with DevOps”: https://learn.microsoft.com/en-us/training/modules/deliver-with-devops/
- GitHub Docs, “About pull requests”: https://docs.github.com/en/pull-requests/collaborating-with-pull-requests
- GitHub Docs, “GitHub Actions documentation”: https://docs.github.com/en/actions
- Microsoft Learn, “Adopt, extend and build Copilot experiences across the Microsoft Cloud”: https://learn.microsoft.com/en-us/microsoft-cloud/dev/copilot/overview

CHAPTER 4

Modern .NET

Chapter Purpose

The first three chapters established the modern .NET landscape, the technology stack, and the development workflow. This chapter moves into the platform at the center of that system: modern .NET itself.

For an experienced developer returning from the .NET 5 era, the important shift is not that C# suddenly became unfamiliar. The shift is that .NET is now a fast-moving, cross-platform, SDK-driven platform with a regular support cadence, simpler project files, stronger command-line workflows, first-class container and cloud integration, and ongoing language evolution.

Modern .NET exists because the old .NET world had reached a boundary. .NET Framework was powerful and deeply integrated with Windows, IIS, Visual Studio, and enterprise infrastructure, but that same integration made it hard to move at cloud speed. The industry was moving toward open source, Linux servers, containers, command-line automation, microservice-friendly runtimes, and frequent platform updates. Java, Node.js, Go, Python, and other ecosystems had already made cross-platform server development feel ordinary.

.NET Core began as Microsoft's answer to that reality: a modular, cross-platform, open-source version of .NET. .NET 5 unified the branding around "one .NET" for modern workloads. By 2026, modern .NET is no longer a side path. It is the main .NET line.

This chapter explains modern .NET versions, SDKs, project structure, cross-platform development, and language improvements. It gives enough practical mechanics to orient you without turning into a full API reference.

Where This Fits

Modern .NET is the application platform layer underneath the rest of the book.

```
Developer workflow
  |
  v
.NET SDK and project system
  |
  v
C# source code, project files, NuGet packages
  |
  v
Build, test, publish
  |
  v
Runtime process
  |
  +-- ASP.NET Core
  +-- worker services
  +-- console tools
  +-- libraries
  |
```

```
  v
Windows, Linux, containers, cloud platforms
```

Chapter 5 builds on this foundation with ASP.NET Core. Chapter 6 applies it to data access. Chapter 8 uses it for automated testing and CI. Chapters 9-13 use it across Linux, Docker, deployment, and cloud hosting.
The key point: modern .NET is both a runtime and a development platform. The runtime runs applications. The SDK builds, tests, publishes, packages, and tools them. The project system describes how source code becomes output.

Connection to the Reader's Existing Model

You already understand .NET as a managed platform: C# compiles to assemblies, the runtime executes managed code, libraries provide reusable APIs, and project files describe how the application is built.
That model still works, but several boundaries have moved.
In .NET Framework, the installed framework felt like part of the Windows machine. Applications targeted a framework version that was installed on the server. Visual Studio was often the center of build and publish behavior. Project files were verbose. IIS was the natural web host. Windows was assumed.
In modern .NET, the SDK is the center of development. The command line can create, build, test, run, publish, and package applications. Visual Studio, Visual Studio Code, Rider, GitHub Actions, Azure Pipelines, Docker builds, and cloud development environments all use the same underlying project and SDK model.
The modern project file is much smaller because SDK-style projects infer many defaults. A project file no longer needs to list every source file by default. NuGet package references are first-class. Target frameworks are explicit.
Side-by-side installation is normal. A machine can have several SDKs and runtimes installed. A repository can use `global.json` to select the SDK range used by command-line builds. This is very different from thinking of the server as having one blessed framework personality.
Cross-platform hosting is normal. A .NET application may be developed on Windows, built on Linux, tested in CI, packaged into a container, and deployed to a managed cloud environment. That does not mean Windows is obsolete. It means Windows is one supported environment rather than the assumption under every layer.
The IIS analogy remains useful for understanding process hosting, ports, configuration, and deployment. It breaks down when you assume IIS owns the application lifecycle. In modern .NET, an ASP.NET Core application is itself a process. IIS may reverse proxy to it, but so may Nginx, a cloud load balancer, a container platform, or a local development server.

Layer 1 — Conceptual Model

Modern .NET is the current, cross-platform .NET implementation for building applications, services, libraries, tools, cloud workloads, desktop apps, mobile apps, games, and AI-enabled systems.
It solves several problems:

- it provides a common managed runtime for C# and other .NET languages;
- it supports cross-platform development and hosting;
- it supplies a consistent SDK and command-line workflow;
- it uses a simpler project format;
- it supports side-by-side versions and regular releases;
- it integrates naturally with packages, tests, containers, cloud platforms, telemetry, and modern tooling.

It does not solve every application concern:

- it does not choose your architecture;
- it does not make your code maintainable automatically;

- it does not remove the need to understand deployment;
- it does not replace SQL Server, Redis, or cloud platforms;
- it does not guarantee cross-platform behavior if your code uses platform-specific APIs;
- it does not make every old .NET Framework application portable without work.

The conceptual center is this:

```
SDK describes how to build.
Project file describes what to build.
Target framework describes which APIs are available.
Runtime executes the built application.
```

Once that model is clear, most modern .NET mechanics become easier to place.

Layer 2 — System Relationships

Modern .NET interacts with the rest of the system through a few important relationships.
The SDK receives project files, source files, package references, target frameworks, analyzers, tool manifests, and build properties. It outputs compiled assemblies, test results, NuGet packages, publish folders, container images in some scenarios, and diagnostic information.
The runtime receives compiled assemblies, dependencies, configuration, command line arguments, environment variables, and operating-system services. It outputs process behavior: HTTP responses, console output, logs, files, network calls, database calls, and exit codes.
The project file owns build intent. It tells the SDK which project SDK to use, which target framework to compile against, which packages to reference, and which build options matter. In SDK-style projects, many conventions are implicit, which keeps the file small but makes it important to understand the defaults.
NuGet owns package distribution. A modern .NET project usually depends on NuGet packages from nuget.org, private feeds, or local package sources. Package restore is therefore part of build reliability and supply-chain security.
The operating system owns process execution, files, environment variables, networking, certificates, native dependencies, and permissions. If an application uses only cross-platform .NET APIs, it can often run on Windows, Linux, and macOS. If it depends on Windows-specific APIs, COM components, registry behavior, GDI, or old framework libraries, portability becomes a design constraint.
The CI system owns clean reproducibility. It uses the SDK to restore, build, test, and publish without relying on a developer's machine. If the build works only inside one person's IDE, the project is not yet modern in workflow terms.
The deployment platform owns runtime placement. It may run the app as a Windows service, IIS-hosted process, Linux systemd service, container, managed app service, serverless function, or orchestrated workload. .NET supports many of these targets, but each target changes configuration and operations.
Failure boundaries include mismatched SDK versions, unsupported target frameworks, missing runtimes, package restore failures, platform-specific code, native dependency issues, incorrect runtime identifiers, broken publish settings, and assuming local development behavior will match production.

Layer 3 — Core Mechanics

The modern .NET mechanics begin with versions.
Microsoft ships major .NET releases annually. Releases alternate between Long Term Support (LTS) and Standard Term Support (STS). As of July 22, 2026, Microsoft lists .NET 10 as an LTS release supported until November 2028, .NET 9 as an STS release supported until November 2026, and .NET 8 as an LTS release supported until November 2026.

That version choice matters. For production enterprise applications, an LTS release is often the conservative default. An STS release may be useful when a team wants newer platform capabilities and accepts a faster upgrade rhythm.
The next mechanic is the distinction between SDK and runtime.
The runtime runs .NET applications. The SDK creates and builds them. A server may need only the runtime if it runs framework-dependent applications. A developer machine and CI agent need the SDK.
The target framework tells the compiler which API surface the project uses. For modern .NET, this appears as a target framework moniker:

```
<TargetFramework>net10.0</TargetFramework>
```

A library can target multiple frameworks when it needs to support more than one consumer:

```
<TargetFrameworks>net8.0;net10.0</TargetFrameworks>
```

Multi-targeting is useful for libraries. It is usually unnecessary for a simple application where the team controls the runtime environment.
The project SDK appears at the top of an SDK-style project file:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

For a web application, the project uses the web SDK:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

`ImplicitUsings` reduces repetitive `using` directives for common namespaces. `Nullable` enables nullable reference type analysis, which helps identify possible null-related bugs at compile time. If you are returning from older C# habits, nullable reference types are one of the most important language-era changes to take seriously.
Runtime identifiers, or RIDs, identify target platforms such as `win-x64`, `linux-x64`, and `osx-arm64`. They matter when publishing for a specific platform or using packages with native assets.
`global.json` can select the SDK version used by CLI commands in a repository:

```
{
  "sdk": {
    "version": "10.0.100",
    "rollForward": "latestFeature"
  }
}
```

This does not choose the runtime your project targets. The target framework does that. `global.json` chooses the SDK used to build.
C# evolves with .NET. As of .NET 10, C# 14 is the latest C# release. It includes features such as extension members, null-conditional assignment, `field` backed properties, improved `Span<T>` conversions, and file-based app support.

You do not need to memorize every new feature immediately. The important production habit is to distinguish language convenience from design clarity.

Layer 4 — Developer Workflow

The modern .NET workflow is SDK-first. You can still use Visual Studio, and many enterprise developers should, but the project should also behave predictably from the command line.
Check your environment:

```
dotnet --info
dotnet --list-sdks
dotnet --list-runtimes
```

Create a console application:

```
dotnet new console -n ModernDotnetDemo
cd ModernDotnetDemo
dotnet run
```

Inspect the project file:

```
cat ModernDotnetDemo.csproj
```

On Windows PowerShell:

```
Get-Content .\ModernDotnetDemo.csproj
```

Build and run:

```
dotnet build
dotnet run
```

Create a solution and add the project:

```
cd ..
dotnet new sln -n ModernDotnetDemo
dotnet sln ModernDotnetDemo.sln add ModernDotnetDemo/ModernDotnetDemo.csproj
```

Create a local tool manifest when a repository needs repeatable local tools:

```
dotnet new tool-manifest
dotnet tool restore
```

Local tools matter because they let a repository define tool dependencies without requiring every developer to install the same global tool manually. Examples later in the book may include formatters, EF Core tools, or other project-specific commands.
Create a `global.json` when a repository needs predictable SDK selection:

```
dotnet new globaljson --sdk-version 10.0.100 --roll-forward latestFeature
```

The exact SDK version should match the version your team supports. In CI, being explicit avoids surprising builds when an agent image updates.
Publish the application:

```
dotnet publish -c Release
```

This produces output that can be run on a machine with an appropriate runtime or packaged into a deployment artifact. Later chapters cover ASP.NET Core, containers, and deployment in detail.

The workflow principle is simple: the same basic commands should work locally, in CI, and in deployment automation.

Layer 5 — Production Usage

Production .NET usage starts with support.

Choose a supported .NET version. For most enterprise production applications, that usually means choosing an LTS version unless a specific STS feature justifies the shorter support window. Keep applications patched with servicing updates because Microsoft support expects supported releases to be on supported servicing levels.

Separate SDK concerns from runtime concerns. Build agents need SDKs. Runtime hosts may need only runtimes, or no preinstalled runtime if you publish self-contained or run in a container image that includes the runtime. Do not assume the production machine has the same SDKs as a developer workstation.

Configuration belongs outside the compiled application. Modern .NET supports configuration from JSON files, environment variables, command-line arguments, secret stores, and platform providers. Local development can use local settings. Production should use mechanisms appropriate to the host platform.

Secrets should not be committed to source control. User secrets can be useful for local development. Production secrets should come from managed secret stores, environment injection, platform identity, or deployment systems.

Reliability depends on predictable builds and repeatable publishes. A production artifact should be identifiable. You should know which commit, target framework, SDK, package versions, and deployment settings produced it.

Observability starts in the application. Even before advanced telemetry, modern .NET applications should emit meaningful logs and expose health where appropriate. ASP.NET Core expands on this in the next chapter.

Cross-platform production requires discipline. File paths, case sensitivity, line endings, certificates, native libraries, time zones, globalization, process signals, and filesystem permissions can differ between Windows and Linux. Code that assumes Windows behavior may compile but still fail when hosted elsewhere.

Cost enters through runtime choices. Self-contained deployments can be larger. Containers introduce base-image and registry management. Cloud hosts charge for compute, memory, storage, networking, and sometimes build minutes. Newer features such as Native AOT can reduce startup time and footprint in some workloads, but they also introduce compatibility and diagnostic tradeoffs.

Local development optimizes for speed and convenience. Production optimizes for supportability, security, repeatability, and clear ownership.

Layer 6 — Tradeoffs and Alternatives

Modern .NET is a strong choice when a team values C#, mature tooling, enterprise libraries, high-performance server workloads, Microsoft ecosystem integration, and cross-platform deployment.

It competes with Java, Go, Node.js, Python, Rust, Kotlin, Ruby, PHP, and other ecosystems. Java remains a major enterprise platform with Spring. Go is common for small, fast cloud-native services and infrastructure tooling. Node.js is natural for JavaScript-centered teams. Python dominates many data and automation workflows. Rust is compelling when memory safety and systems-level control matter. Modern .NET's advantage is the combination of C#, runtime performance, enterprise fit, tooling, and platform breadth.

Use modern .NET for new .NET applications unless you have a specific reason not to. Use .NET Framework only when the application depends on technologies that require it, such as some legacy ASP.NET, WCF server scenarios, Windows-specific desktop dependencies, or older third-party libraries that cannot move.

Choose LTS for conservative production systems. Choose STS only when the team has an explicit upgrade plan and wants newer features sooner.

Use SDK-style projects for modern projects. Avoid manually expanding project files into old-style file lists unless a special build requirement demands it.

Use multi-targeting for reusable libraries that must support multiple consumers. Avoid multi-targeting applications without a clear reason; it increases testing and support work.
Use self-contained deployment when you need to carry the runtime with the app. Use framework-dependent deployment when the host environment manages the runtime. Use containers when repeatable packaging and runtime environment consistency matter.
Use Native AOT selectively. It can improve startup time and deployment size for some workloads, especially small services and command-line tools, but it can limit reflection-heavy libraries and change diagnostics.
Common overengineering mistakes:

- upgrading target frameworks without checking package and hosting support;
- treating every new C# feature as a reason to rewrite old code;
- multi-targeting applications unnecessarily;
- pinning SDK versions so tightly that servicing becomes painful;
- ignoring nullable warnings because the project still compiles;
- assuming cross-platform means every API behaves identically everywhere;
- choosing self-contained, containerized, trimmed, and AOT deployment all at once before understanding the tradeoffs.

The state-of-the-art direction in modern .NET is pragmatic platform breadth: fast runtime, minimal hosting, container support, cloud integration, better diagnostics, optional Native AOT, strong language evolution, and AI-friendly libraries. The skill is knowing which capabilities the application actually needs.

Layer 7 — Interview Perspective

Interviewers use modern .NET questions to test whether you understand the platform model, not just C# syntax.
Concepts commonly tested:

- .NET Framework versus .NET Core versus modern .NET;
- LTS versus STS releases;
- SDK versus runtime;
- target frameworks and target framework monikers;
- SDK-style project files;
- NuGet package references;
- `global.json`;
- framework-dependent versus self-contained deployment;
- cross-platform hosting;
- nullable reference types;
- when to use newer language features.

Representative questions:

- “What changed between .NET Framework and modern .NET?”
- “What is the difference between the .NET SDK and the runtime?”
- “Why would a project use `global.json`?”
- “What does `net10.0` mean?”
- “When would you choose an LTS release?”
- “What makes SDK-style project files different from older project files?”
- “What should you check before running a .NET application on Linux?”
- “When would self-contained deployment be useful?”

A strong answer connects versioning, build, and runtime:

> "The target framework controls which APIs the project compiles against. The SDK builds the project, and `global.json` can influence which SDK is selected for CLI commands. The runtime is what actually executes the application. In production, those choices affect support, deployment size, patching, and compatibility."

Common misconceptions:

- ".NET is just a renamed .NET Framework."
- "The SDK and runtime are the same thing."
- "`global.json` chooses the target framework."
- "If it compiles on Windows, it is automatically Linux-ready."
- "LTS means the application never needs patching."
- "New C# syntax is always a net improvement."
- "Self-contained deployment is always better."

Small design scenario:
You inherit a .NET 5 internal API. It builds only in Visual Studio, has no `global.json`, targets an out-of-support framework, and is deployed manually to a Windows server.
A good modernization plan would:

- move to a supported LTS target framework;
- verify package compatibility;
- make the build work through `dotnet build`;
- add or update `global.json` if the team needs SDK predictability;
- enable nullable analysis thoughtfully if it is not already enabled;
- define a repeatable publish command;
- confirm whether the app has Windows-specific assumptions;
- defer containers, cloud migration, and advanced deployment until the platform baseline is healthy.

The strong answer upgrades the foundation before changing the hosting model.

Hands-On Lab

Objective:
Create a small modern .NET console application and inspect the SDK-style project model.
Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- A terminal.
- A text editor or IDE.

Steps:

1. Create a working folder:

```
mkdir ModernDotnetLab
cd ModernDotnetLab
```

2. Check installed SDKs and runtimes:

```
dotnet --info
dotnet --list-sdks
dotnet --list-runtimes
```

3. Create a console app:

```
dotnet new console -n HelloModernDotnet
cd HelloModernDotnet
```

4. Open `HelloModernDotnet.csproj` and identify:
   - the project SDK;
   - the target framework;
   - whether nullable reference types are enabled;
   - whether implicit usings are enabled.
5. Run the app:

```
dotnet run
```

6. Build in Release configuration:

```
dotnet build -c Release
```

7. Publish the app:

```
dotnet publish -c Release
```

8. Create a solution one directory above the project:

```
cd ..
dotnet new sln -n ModernDotnetLab
dotnet sln ModernDotnetLab.sln add HelloModernDotnet/HelloModernDotnet.csproj
```

9. Create a `global.json`:

```
dotnet new globaljson --roll-forward latestFeature
```

10. Re-run:

```
dotnet --info
```

Expected results:

- You can identify the SDK and runtime versions installed.
- You can explain the project SDK and target framework in the project file.
- You can build, run, and publish from the command line.
- You can explain what `global.json` does and does not control.

Validation commands:

```
dotnet --info
dotnet build
dotnet run
dotnet publish -c Release
```

Troubleshooting notes:

- If `dotnet` is not recognized, install the .NET SDK and reopen the terminal.
- If `dotnet new console` targets a different framework than expected, inspect which SDK is installed and selected.
- If `global.json` causes an SDK error, the requested SDK may not be installed or the roll-forward setting may be too restrictive.
- If publish output is hard to find, inspect the `bin/Release` folder under the project.

Knowledge Check

1. Why did modern .NET move away from the Windows-centered assumptions of .NET Framework?
2. What is the difference between the .NET SDK and the .NET runtime?
3. Why does the target framework matter to both compilation and production support?
4. When is an LTS release usually preferable to an STS release?
5. What does an SDK-style project file hide through convention?
6. What does `global.json` control, and what does it not control?
7. Why can cross-platform development still fail when the code compiles?
8. When would multi-targeting be useful?
9. What production tradeoff does self-contained deployment make?
10. Why should nullable reference type warnings be treated as design feedback instead of noise?

Summary

Modern .NET is the main .NET platform for current professional development. It grew out of the need for a cross-platform, open-source, cloud-ready, SDK-driven successor to the Windows-centered .NET Framework world.
The central mechanics are straightforward once the layers are separated. The SDK builds and tools applications. The runtime executes them. The project file describes build intent. The target framework describes the API surface. NuGet packages provide dependencies. `global.json` can select the SDK used by command line builds. Runtime identifiers describe target platforms when platform- specific publishing or native assets matter.
Modern .NET's power is not only newer APIs. It is that the same project model can participate in local development, command-line builds, CI, containers, Linux hosting, cloud platforms, and production deployment.
The next chapter builds on this foundation with ASP.NET Core, where modern .NET becomes a web and API application platform.

Sources

- Microsoft Learn, ".NET releases, patches, and support": https://learn.microsoft.com/en-us/dotnet/core/releases-and-support
- Microsoft Learn, "What's new in .NET 10": https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview
- Microsoft Learn, ".NET project SDKs": https://learn.microsoft.com/en-us/dotnet/core/project-sdk/overview
- Microsoft Learn, "Target frameworks in SDK-style projects": https://learn.microsoft.com/en-us/dotnet/standard/frameworks
- Microsoft Learn, ".NET Runtime Identifier catalog": https://learn.microsoft.com/en-us/dotnet/core/rid-catalog
- Microsoft Learn, "global.json overview": https://learn.microsoft.com/en-us/dotnet/core/tools/global-json
- Microsoft Learn, "Install .NET on Windows": https://learn.microsoft.com/en-us/dotnet/core/install/windows
- Microsoft Learn, "What's new in C# 14": https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/csharp-14

Further Reading

- Microsoft Learn, ".NET CLI overview": https://learn.microsoft.com/en-us/dotnet/core/tools/
- Microsoft Learn, ".NET tools": https://learn.microsoft.com/en-us/dotnet/core/tools/global-tools

- Microsoft Learn, “C# language reference”: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/
- Microsoft Learn, “.NET application publishing overview”: https://learn.microsoft.com/en-us/dotnet/core/deploying/